Lab 4

Name: Mahdi Davoodzadeh
SN: 405725536
Repository: https://github.com/deep-observability-lab/Linux-Kernel-Tracing/tree/main/LABS/LAB1

1. Objective

In this assignment, I used `hping3` to perform separate TCP SYN and UDP scans against `8.8.8.8` on port 53. I first observed the responses from each scan, then used `perf stat` to identify the network-related kernel tracepoints that were triggered. Finally, I recorded the events with `perf record`, inspected their call stacks, and selected `tcp:tcp_send_reset` for a detailed comparison of the two scan methods.

I recorded the SYN and UDP scans separately and sent three probe packets in each run.

2. Environment and Preparation

I performed this assignment on Ubuntu 24.04.5 LTS with kernel `6.17.0-40-generic` and Perf version `6.17.13`. I installed `hping3` and organized the experiment inside the `lab4` directory.

I used `scripts/01_network_preflight.sh` to check that the tools were installed, confirm the route to `8.8.8.8`, and discover available tracepoints. The route used network interface `wlp0s20f3`. The relevant tracepoint groups were available:

Tracepoint group	Number of event directories
net	16
skb	3
tcp	24
udp	1
sock	7

I confirmed the availability of `net:net_dev_queue`, `net:net_dev_xmit`, `net:netif_receive_skb`, `skb:kfree_skb`, `tcp:tcp_send_reset`, and `udp:udp_fail_queue_rcv_skb`. These gave me candidate events to examine during the two scans.

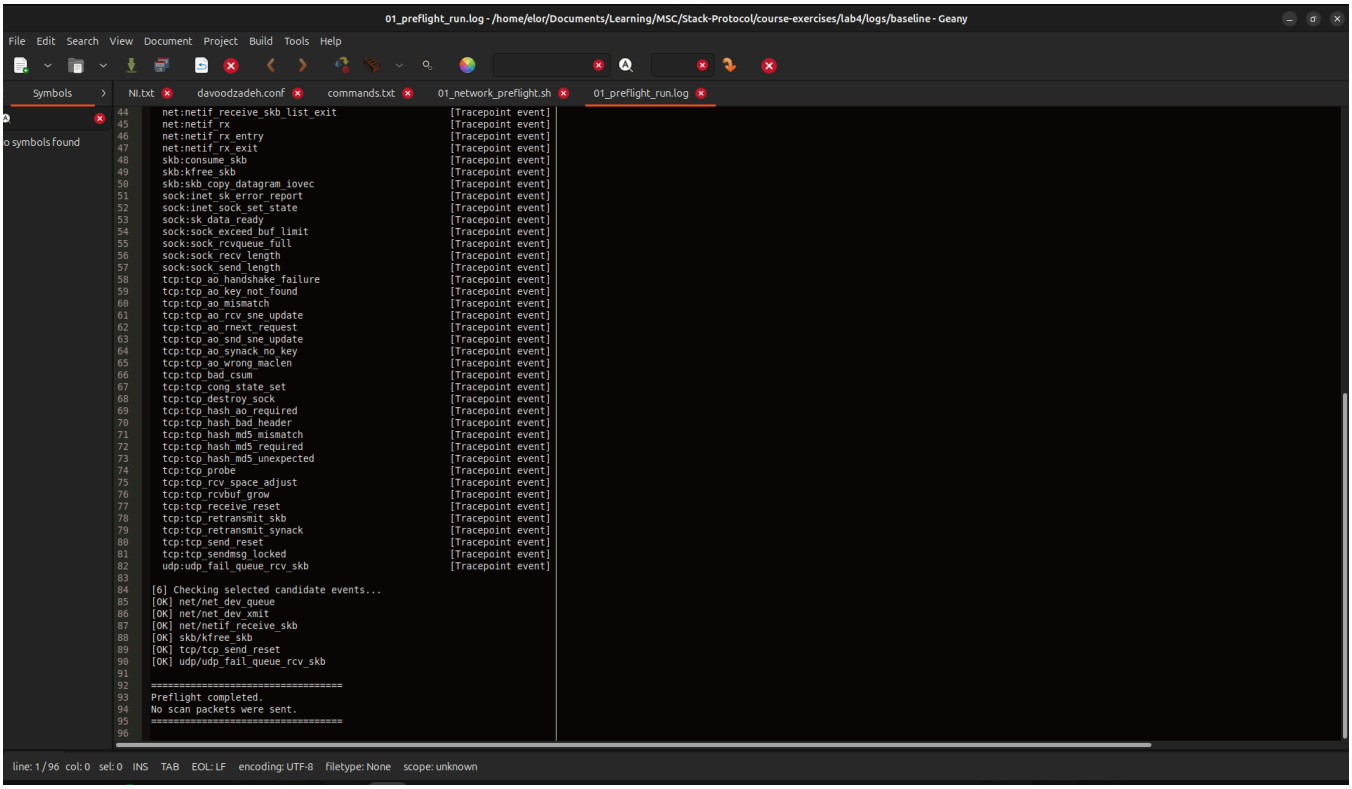


Figure 1 — Initial discovery and availability checks for the network-related tracepoints.

3. TCP SYN Scan

I used `scripts/02_hping_syn_scan.sh` to send three TCP SYN packets to `8.8.8.8` on destination port 53, with a 500 ms interval between packets. The scan was equivalent to:

```
sudo hping3 --syn --destport 53 --count 3 \
  --interval u500000 --numeric 8.8.8.8
```

The program reported three responses, all containing the `SA` flags:

```
HPING 8.8.8.8 (wlp0s20f3 8.8.8.8): S set, 40 headers + 0 data bytes
len=44 ip=8.8.8.8 ttl=114 DF id=0 sport=53 flags=SA seq=0 win=65535 rtt=219.0 ms
len=44 ip=8.8.8.8 ttl=114 DF id=0 sport=53 flags=SA seq=1 win=65535 rtt=229.5 ms
len=44 ip=8.8.8.8 ttl=114 DF id=0 sport=53 flags=SA seq=2 win=65535 rtt=242.2 ms
```

The summary was **3 packets transmitted, 3 packets received, and 0% packet loss**, with an average RTT of **230.3 ms**. The `SA` flags indicate SYN-ACK replies, which are consistent with an accessible, open TCP port 53 at the time of the scan. I used the SYN-ACK replies to assess the TCP port, then examined the local reset behavior in the Perf recording.

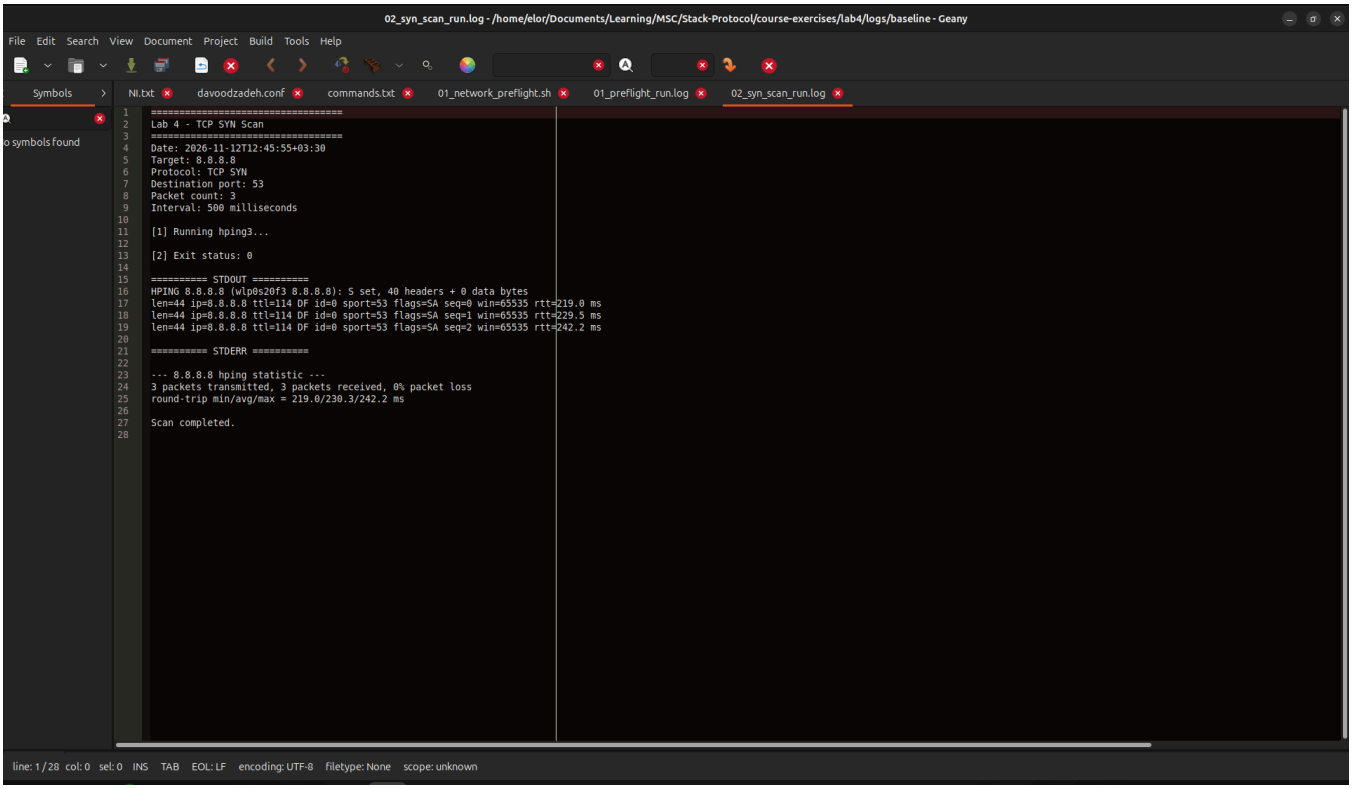


Figure 2 — All three TCP SYN probes received SYN-ACK responses from the destination.

4. UDP Scan with a DNS Payload

I then used `scripts/03_hping_udp_scan.sh` to send three UDP packets to the same destination and port. I used a **29-byte DNS query** for an A record of `example.com`. This made the payload relevant to the DNS service normally associated with UDP port 53.

The payload was stored in `payloads/dns_query.bin`. The important `hping3` arguments were:

```
sudo hping3 --udp --destport 53 --count 3 \
  --interval u500000 --numeric \
  --data 29 --file payloads/dns_query.bin 8.8.8.8
```

The observed output was:

```
HPING 8.8.8.8 (wlp0s20f3 8.8.8.8): udp mode set, 28 headers + 29 data bytes
len=89 ip=8.8.8.8 ttl=114 id=63379 seq=0 rtt=190.9 ms
len=89 ip=8.8.8.8 ttl=114 id=8359 seq=1 rtt=201.5 ms
len=89 ip=8.8.8.8 ttl=114 id=40700 seq=2 rtt=214.4 ms
```

This scan also reported **3 packets transmitted, 3 received, and 0% packet loss**, with an average RTT of **202.3 ms**. The output confirmed replies from the destination during the UDP scan. I compared their packet counts and reported round-trip times.

The run log included a `memlockall()` warning about disabling memory paging. The scan completed with exit status 0 and three received replies.

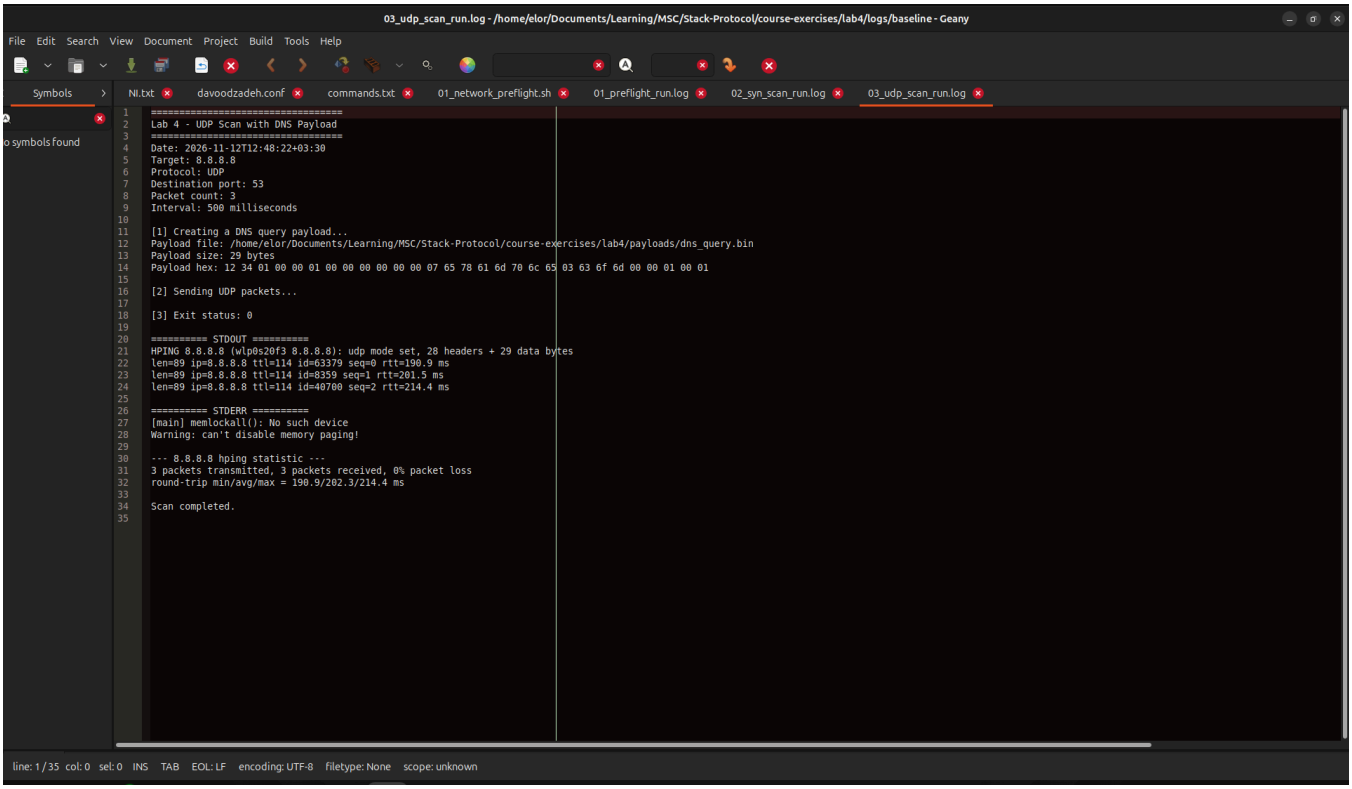


Figure 3 — Three UDP probes with a 29-byte DNS payload and their observed replies.

Initial Comparison

Measurement	TCP SYN	UDP
Destination	`8.8.8.8:53`	`8.8.8.8:53`
Probe packets sent	3	3
Responses reported	3	3
Packet loss reported	0%	0%
Probe payload	None	29-byte DNS query
Average baseline RTT	230.3 ms	202.3 ms
Response observed	TCP SYN-ACK (`SA`)	Replies in UDP scan mode

The RTT values describe these separate, three-packet runs and their respective protocol exchanges.

5. Identifying Triggered Events with `perf stat`

I wrote `scripts/04_perf_network_stat.sh` to compare relevant kernel tracepoints. I used `perf stat -a` so that I could observe network processing occurring outside the `hping3` process, such as receive-side work performed by interrupt threads or softirqs. I collected an idle reference interval, followed by separate SYN and UDP scans.

The recorded counts were:

Kernel event	Idle (2.004 s)	TCP SYN (1.271 s)	UDP (1.340 s)
net:net_dev_queue	4	6	6
net:net_dev_start_xmit	4	6	6
net:net_dev_xmit	4	6	6
net:netif_receive_skb	5	3	3
skb:kfree_skb	0	3	3
tcp:tcp_send_reset	**0**	**3**	**0**
udp:udp_fail_queue_rcv_skb	0	0	0
sock:inet_sock_set_state	0	0	0

The event counts indicated that both scan methods exercised the network transmit and receive paths. The clearest protocol-specific difference was `tcp:tcp_send_reset`: I observed **three** events during the SYN scan but **zero** during the UDP scan.

These system-wide counts include background traffic. I compared each run with its recorded duration and used the idle interval as a reference for background activity.

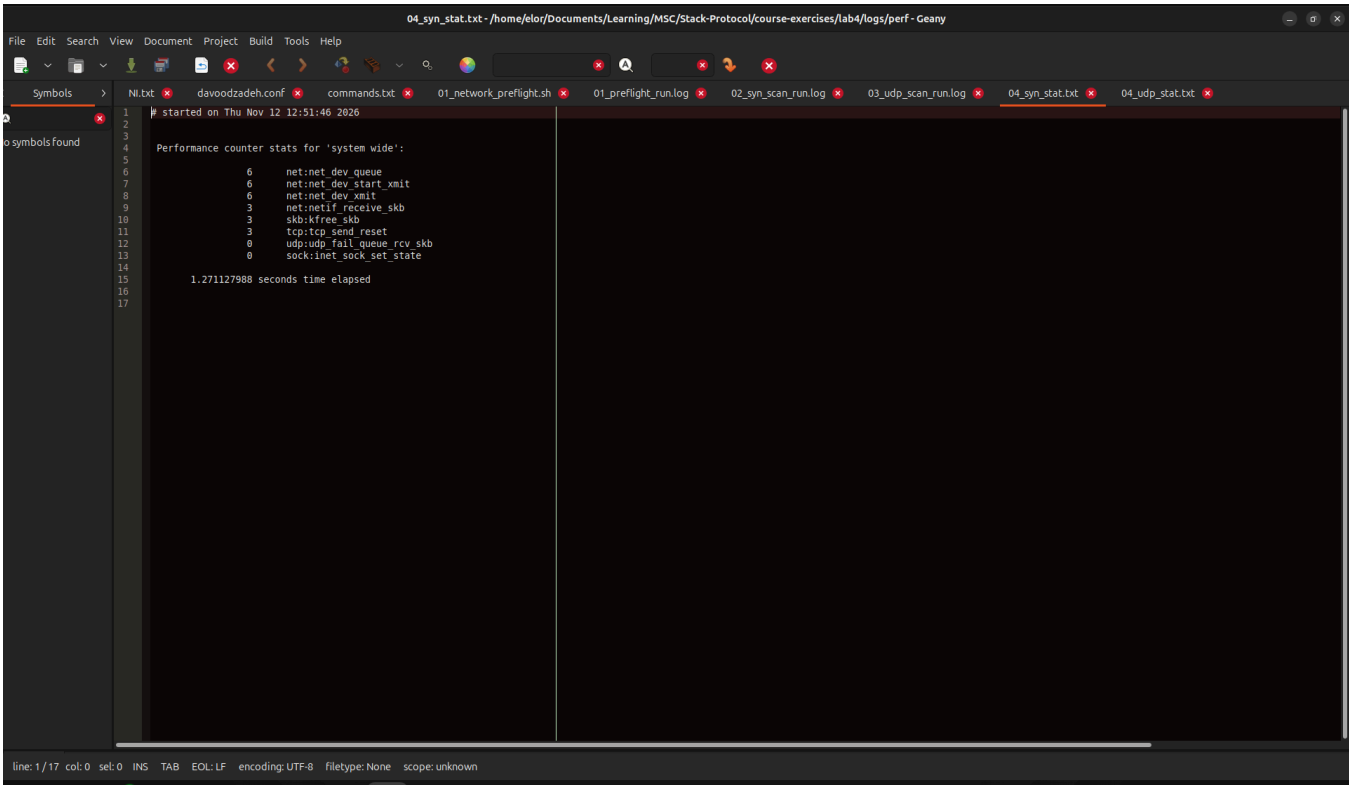


Figure 4 — Perf statistics for the TCP SYN scan. The system-wide capture recorded three TCP reset events over 1.271 seconds.

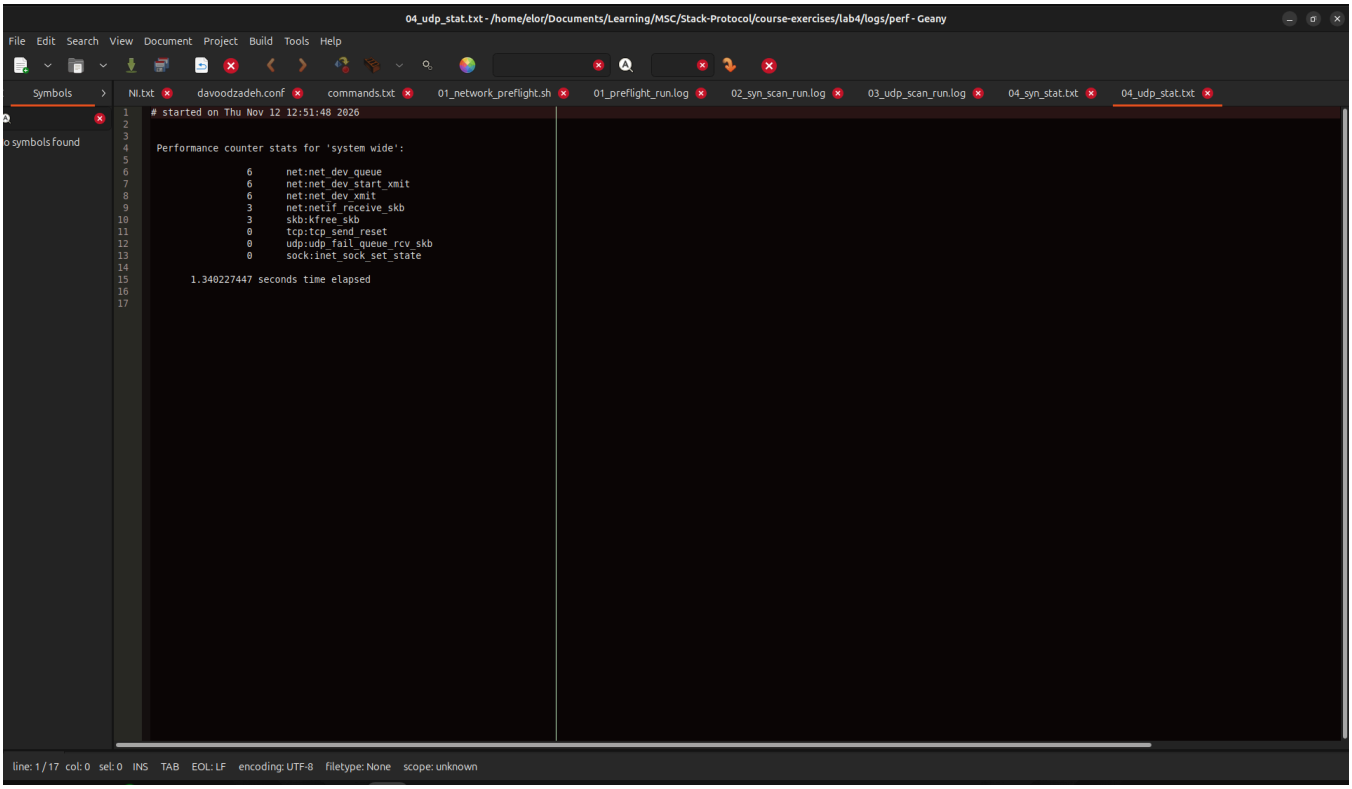


Figure 5 — Perf statistics for the UDP scan. The system-wide capture recorded zero TCP reset events over 1.340 seconds, alongside transmit and receive activity.

6. Recording Network Events with `perf record`

To investigate the events more deeply, I created `scripts/05_perf_network_record.sh`. I recorded the following tracepoints in separate system-wide Perf sessions with frame-pointer call graphs:

```
net:net_dev_queue
net:net_dev_xmit
net:netif_receive_skb
skb:kfree_skb
tcp:tcp_send_reset
```

The recordings used `perf record -a --call-graph fp`, and I exported the samples with `perf script`. The binary recordings were saved as `logs/perf/05_syn_perf.data` and `logs/perf/05_udp_perf.data`.

Recorded event	SYN recording	UDP recording
`net:net_dev_queue`	10	27
`net:net_dev_xmit`	10	27
`net:netif_receive_skb`	8	29
`skb:kfree_skb`	3	3
`tcp:tcp_send_reset`	**3**	**0**

The exported SYN trace had **1,151 lines** and the UDP trace had **3,688 lines**.

The `perf stat` and `perf record` results describe separate system-wide captures. General network-event totals therefore include the background activity present during each run. I used the reset events' address and port fields to associate the selected events with the SYN probes.

7. Detailed Comparison of the Selected Event: `tcp:tcp_send_reset`

I selected **`tcp:tcp_send_reset`** because its count differed between the two scan methods and its tracepoint fields allowed me to verify the relevant IP addresses and ports.

I used `scripts/06_compare_syn_udp.sh` to inspect both exported Perf traces. It found:

Measurement	SYN	UDP
Total `tcp:tcp_send_reset` events	3	0
Reset events referencing `8.8.8.8:53`	3	0

The three SYN events recorded these local source ports: **2435**, **2436**, and **2437**. Their destination was `8.8.8.8:53`. Each event included:

```
state=UNKNOWN reason=NO_SOCKET
```

The `NO_SOCKET` reason identifies a reset generated for an incoming packet with no matching local TCP socket. This is consistent with the raw TCP SYN probes generated by `hping3`.

7.1 Why the SYN Scan Triggered TCP Resets

I observed SYN-ACK replies (`flags=SA`) from the destination. When these replies reached my computer, the local kernel passed them through its TCP receive path. The reply to the raw probe reached the `NO_SOCKET` handling path, where the kernel generated a TCP reset.

The following excerpt shows selected functions from the recorded call stack in calling order. The original `perf script` output lists the most recent function first:

```
handle_softirqs
  -> net_rx_action
    -> __napi_poll
      -> iwl_pcie_napi_poll_msix
        -> napi_complete_done
          -> netif_receive_skb_list_internal
            -> __netif_receive_skb_list_core
              -> ip_list_rcv
                -> ip_local_deliver
                  -> ip_protocol_deliver_rcu
                    -> tcp_v4_rcv
                      -> tcp_v4_send_reset
                        -> perf_trace_tcp_send_reset
```

The stack connects softirq processing, the Wi-Fi receive path, IP delivery, and TCP reset generation. Both `tcp_v4_rcv` and `tcp_v4_send_reset` appear in the recorded stack. Perf attributed the reset events to `irq/...-iwlwifi` threads, showing the receive-side execution context for this run.

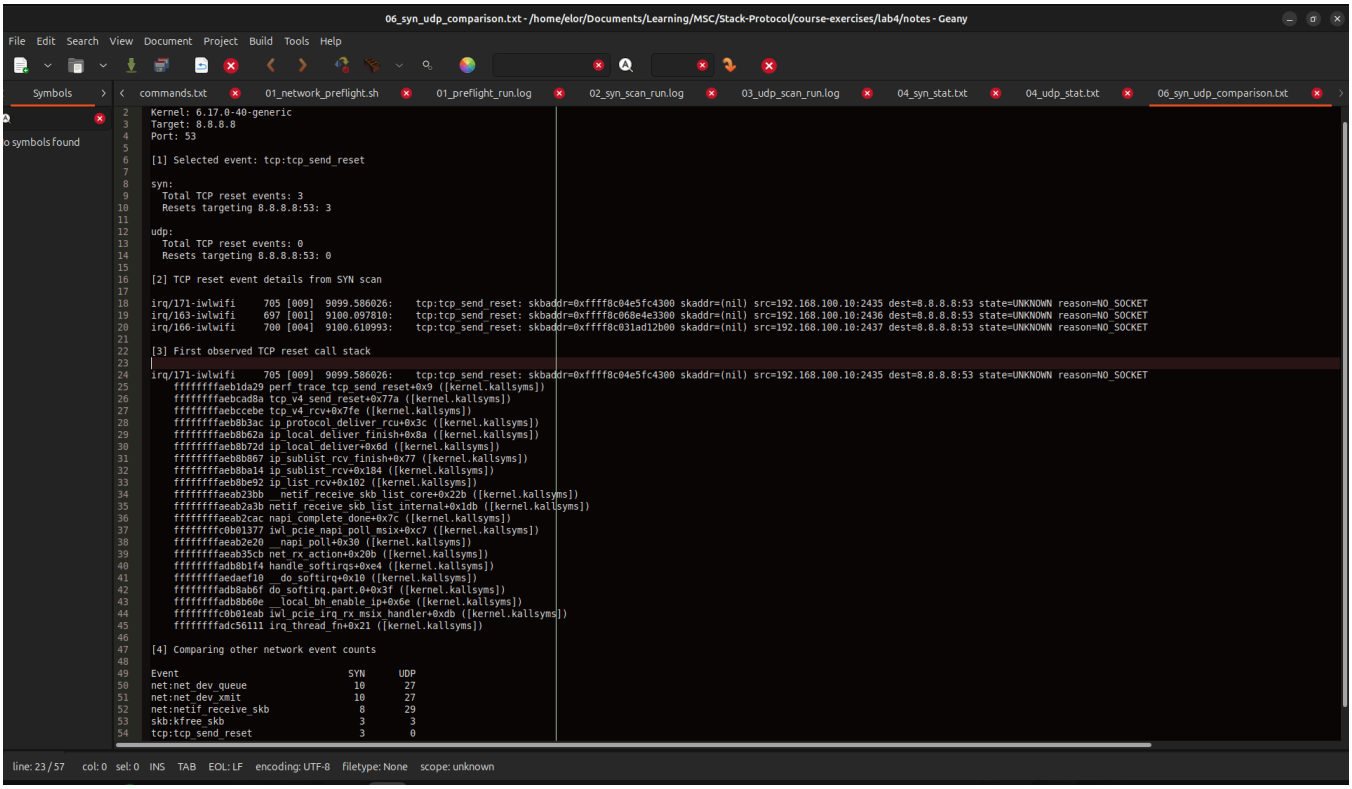


Figure 6 — TCP reset comparison and call stack. The three reset events target `8.8.8.8:53` and carry `reason=NO_SOCKET`. The first event's stack contains `tcp_v4_rcv` and `tcp_v4_send_reset` in a Wi-Fi IRQ thread. The UDP recording contains zero reset events.

7.2 Network Activity During the UDP Scan

My UDP scan sent a DNS query and received replies using UDP's connectionless exchange. The recorded activity included network transmission, reception, and packet-buffer freeing.

The UDP recording contained **27** `net:net_dev_queue` events, **27** `net:net_dev_xmit` events, **29** `net:netif_receive_skb` events, **3** `skb:kfree_skb` events, and **zero** `tcp:tcp_send_reset` events. The SYN recording contained three resets associated with `8.8.8.8:53`. This comparison links the TCP-specific reset activity to the raw SYN probe exchange.